

Plan de réalisation — PIM / DAM

Business Profilers — Référentiel prestataires (MDM) · Juillet 2026 · Version de cadrage simple

Rapport généré par Fable 5 et GPT 5.6 depuis mes notes et schémas.

Objectif du document : présenter une orientation générale, simple et évolutive pour le futur PIM/DAM. Ce document n’est ni un plan de développement détaillé, ni un chiffrage, ni un engagement contractuel. Les options signalées « à discuter » restent soumises à validation.

1. L'idée en bref

- **Un seul projet Symfony** (monolithe modulaire), déployé sur Upsun dans un projet et sur des ressources d’hébergement distincts de la Marketplace : une app web (PIM, API, back-office) et des workers pour le DAM et les traitements asynchrones.
- **Isolation de la Marketplace** : le PIM/DAM ne partage pas son serveur applicatif, sa base ni ses workers avec la Marketplace. Une panne ou une saturation propre à l’un des projets ne doit donc pas arrêter l’autre.
- **Service partagé en V1 : MariaDB**, pour les données métier et les files Symfony Messenger via le transport Doctrine.
- **Infrastructure V1 réduite à deux fournisseurs** : Upsun (apps et MariaDB) + OVH (stockage S3 et CDN). RabbitMQ, Redis et OpenSearch sont différés en V2.
- **Médias sur OVH Object Storage S3** : bucket privé dam-originals pour les fichiers d’origine, bucket dam-public pour les rendus optimisés et CDN devant les rendus publiés.
- **Les traitements lourds restent asynchrones** avec Symfony Messenger : images, indexation interne, traduction, IA, mails et synchronisation Salesforce. En V1, les messages sont stockés dans MariaDB par le transport Doctrine.
- **Modèle de données** : entités Doctrine fixes par typologie et registre d’attributs pour la gouvernance. Pas de champs dynamiques de type MaPS/EAV.
- **Recherche V1** : MariaDB, index composites et fulltext. OpenSearch sera ajouté uniquement si les mesures de production le justifient.

Principe directeur : commencer avec le minimum d’infrastructure nécessaire au volume actuel, mesurer les usages réels, puis ajouter les briques spécialisées sans réécrire le métier. Symfony Messenger, Cache et les interfaces de recherche permettent cette évolution progressive.

2. Stack technique

Brique	Choix	Rôle
Framework	Symfony + API Platform	Back-office, API REST, import/export
Base de données	MariaDB	Données des fiches, audit, sessions et stratégie d’index définie dès la conception
Files V1	Symfony Messenger + transport Doctrine	Traitements asynchrones durables, retries et files logiques par domaine, sans service supplémentaire
Files V2	RabbitMQ	À activer si le débit, les backlogs ou la multiplication des workers dépassent ce que le transport Doctrine absorbe confortablement
Cache V2	Redis	V1 : cache Symfony filesystem/APCu et sessions en base ; Redis si un cache partagé ou un rate limiting distribué devient nécessaire

2. Stack technique — suite

Brique	Choix	Rôle
Stockage médias	Flysystem + OVH S3	Deux espaces logiques : originaux privés et rendus optimisés
Hébergement	Upsun, projet dédié	Web, workers, crons, base et environnements sur des ressources distinctes de la Marketplace afin d'éviter un point de panne applicatif commun
IA	Couche d'abstraction	LLM, vision, OCR, Google Business Profile et scraping, appelés uniquement par les workers
Dédoublonnage images	Hash exact + pHash	Détection non bloquante dans le pipeline média
Recherche V2	OpenSearch, optionnel	À introduire pour les facettes complexes, la recherche multilingue avancée ou des lenteurs mesurées

Pourquoi conserver Upsun ?

Upsun est déjà connu et utilisé par l'équipe. Le conserver réduit le temps d'apprentissage, facilite l'exploitation quotidienne et permet de réutiliser les pratiques existantes de déploiement, d'environnements et de supervision.

« Les workers exécutent le même code que l'application web dans des conteneurs indépendants, et leur processus est automatiquement redémarré s'il s'arrête. »

Documentation Upsun — [Configurer des workers Symfony](#)

Concrètement, les consumers Symfony Messenger sont déclarés dans la configuration Upsun avec leur propre commande et leurs propres ressources. Ils restent séparés du conteneur web tout en utilisant exactement le même build applicatif, ce qui correspond bien aux besoins du DAM, des imports, des synchronisations et de l'IA.

3. Architecture modulaire — les « bounded contexts »

Un bounded context est un module métier autonome à l'intérieur du même code. Il possède ses entités, ses tables, ses messages et ses handlers. Les échanges inter-modules passent par des messages Messenger et par quelques interfaces stables définies dans Shared/.

```
src/
├─ Pim/           → fiches, 5 typologies, statuts, complétude, audit, API
├─ Dam/           → médias, validation, rendus, pHash, tagging, OCR, S3
├─ Etl/           → imports/exports, Salesforce, webhooks, publications
├─ Enrichment/    → IA, traduction, extraction, GMB, scraping
├─ Account/       → utilisateurs, rôles, affiliations
└─ Shared/        → messages, value objects et interfaces
```

Règles du découpage

- Chaque module a ses propres tables et une file Messenger logique dédiée. En V1, les files utilisent le transport Doctrine/MariaDB ; les workers restent séparés par domaine afin qu'un backlog IA ne bloque pas le pipeline média.
- Le PIM envoie au DAM un message MediaUploaded contenant la clé S3, le mediaId, le checksum et les rendus attendus. Il reçoit ensuite un MediaProcessed contenant les URLs, tags, statut et éventuel doublon.
- Les handlers et callbacks sont idempotents sur leurs identifiants métier : un message rejoué ne crée pas de doublon.
- Le code dépend des abstractions Messenger, Cache et Search, jamais directement de RabbitMQ, Redis ou OpenSearch. Leur ajout en V2 devient alors une évolution d'infrastructure et de configuration, pas une refonte fonctionnelle.

Évolution préparée : le passage de Doctrine à RabbitMQ se fait par changement de transport Messenger et déploiement des consumers adaptés. Les noms de messages, handlers et règles métier restent les mêmes.

4. Pourquoi une V1 sans RabbitMQ, Redis ni OpenSearch ?

La cible du cahier des charges atteint 200 000 fiches et 1 000 000 de médias, mais cette volumétrie n’est pas celle du démarrage. Ajouter immédiatement trois services distribués augmenterait le coût, l’exploitation, la supervision et les scénarios de panne avant d’apporter un bénéfice mesurable.

RabbitMQ différé

Symfony Messenger fonctionne dès la V1 avec le transport Doctrine. MariaDB conserve les messages, les retries et les files d’échec. Pour le volume actuel et une concurrence maîtrisée, cela suffit à découpler l’enregistrement utilisateur des traitements lourds.

Redis différé

Le cache natif Symfony, APCu/filesystem et les sessions en base couvrent une première phase. Redis devient utile pour partager un cache très sollicité entre plusieurs instances, centraliser certains verrous ou appliquer un rate limiting distribué fin.

OpenSearch différé

MariaDB correctement indexée permet la recherche par ID, nom, adresse et les filtres courants. OpenSearch devient pertinent pour les facettes complexes, la pertinence multilingue avancée ou lorsque les mesures montrent que SQL ne tient plus les objectifs.

Pourquoi cette approche accélère le développement

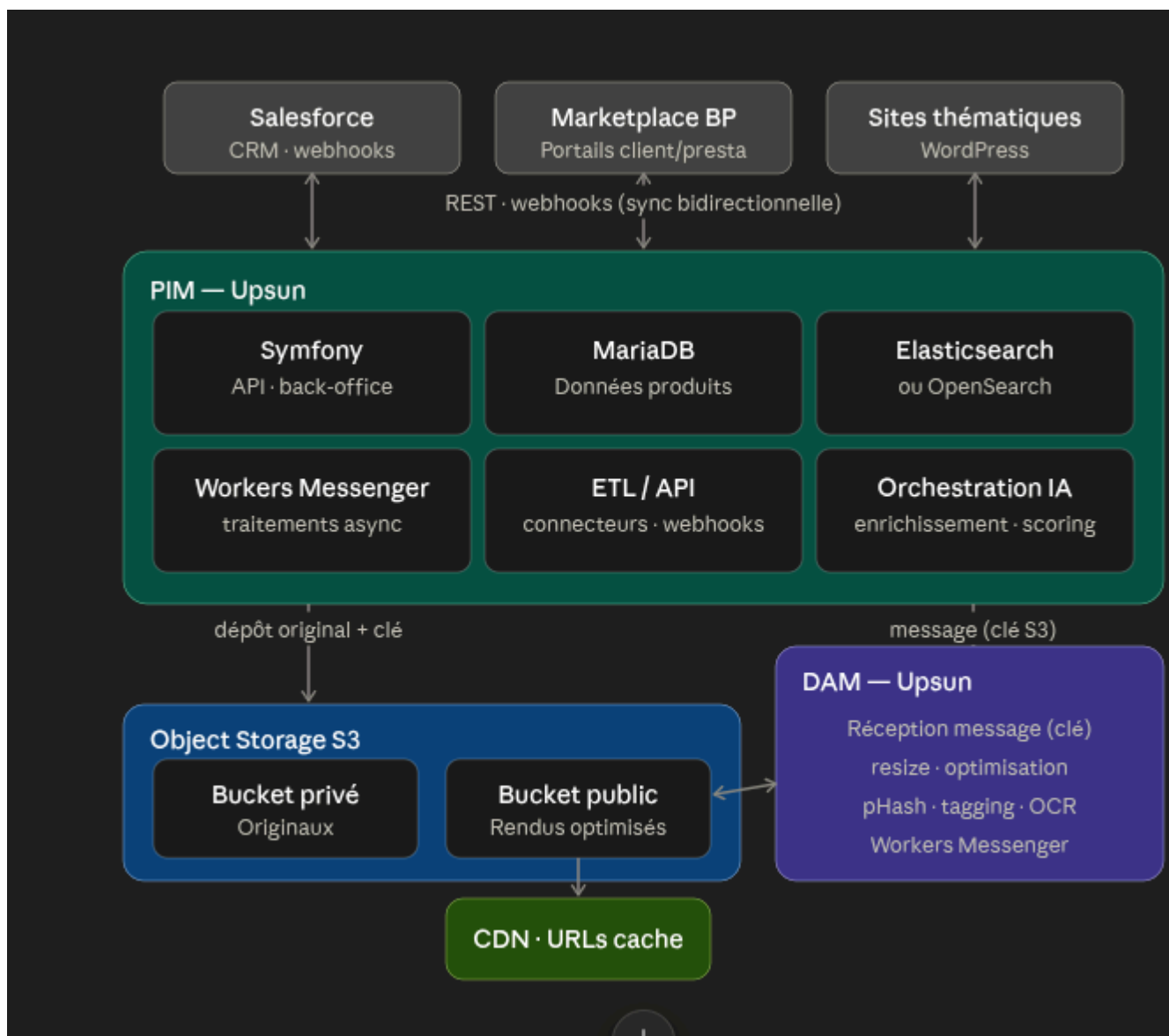
- Un seul service de données à installer, sécuriser, sauvegarder et superviser en V1.
- Des environnements de développement et de CI plus simples et plus proches les uns des autres.
- Moins de temps consacré à l’infrastructure avant d’avoir validé les parcours, le modèle de données et les synchronisations.
- Des coûts initiaux réduits et une capacité à dimensionner les services V2 à partir de métriques réelles.

Déclencheurs mesurables de passage en V2

Service	Signaux d’activation	Préparation réalisée en V1
RabbitMQ	Backlogs persistants, forte concurrence d’écriture sur la table Messenger, besoin de routage avancé ou de montée en charge indépendante des workers	Messages typés, handlers idempotents, files et workers séparés par domaine
Redis	Plusieurs instances web, taux de cache insuffisant, verrous distribués ou rate limiting global à fort débit	Utilisation des contrats Symfony Cache, Lock, Session et RateLimiter
OpenSearch	Temps de réponse SQL hors objectif, facettes complexes, analyse linguistique ou classement de pertinence avancé	Interface de recherche et événements d’indexation indépendants du moteur

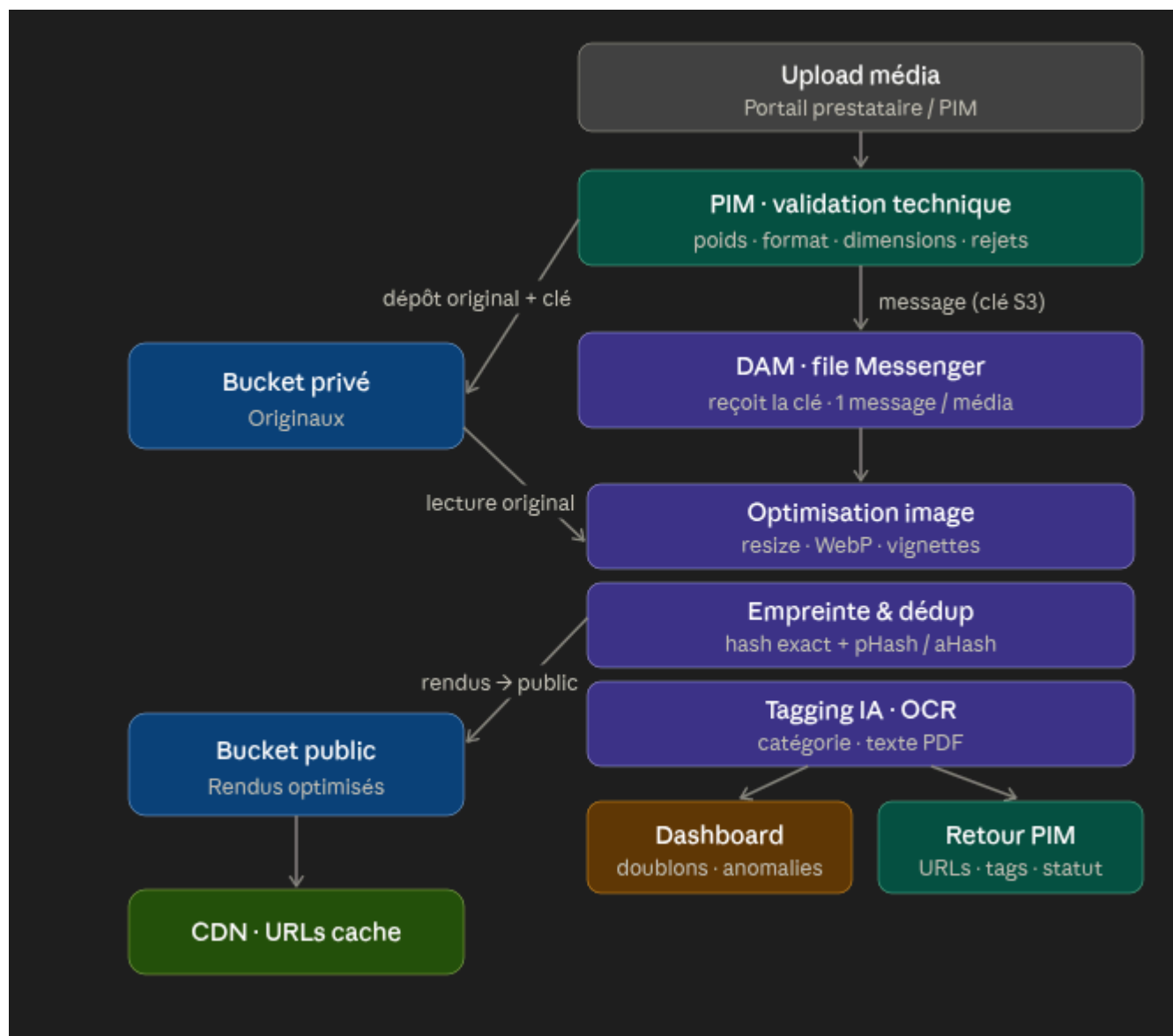
Conclusion : RabbitMQ, Redis et OpenSearch ne sont pas supprimés de l’architecture. Ils deviennent des capacités V2 prévues dès la conception, mais installées seulement lorsque la volumétrie et les mesures de production en démontrent le besoin.

5. Schémas d'architecture



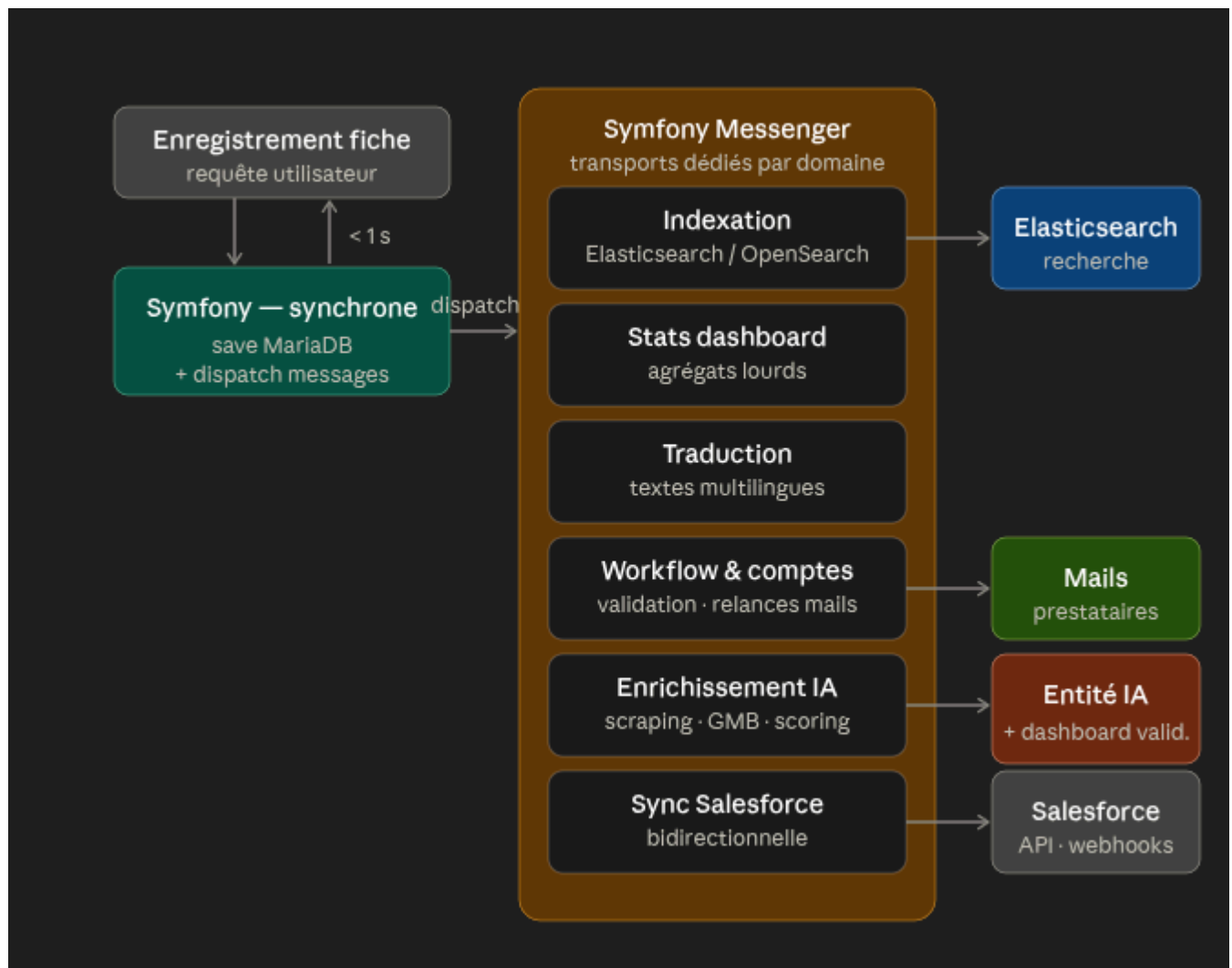
Vue logique PIM / DAM. En V1, MariaDB assure aussi le transport Messenger Doctrine. Elasticsearch/OpenSearch représente une capacité optionnelle V2.

5. Schémas d'architecture — pipeline média



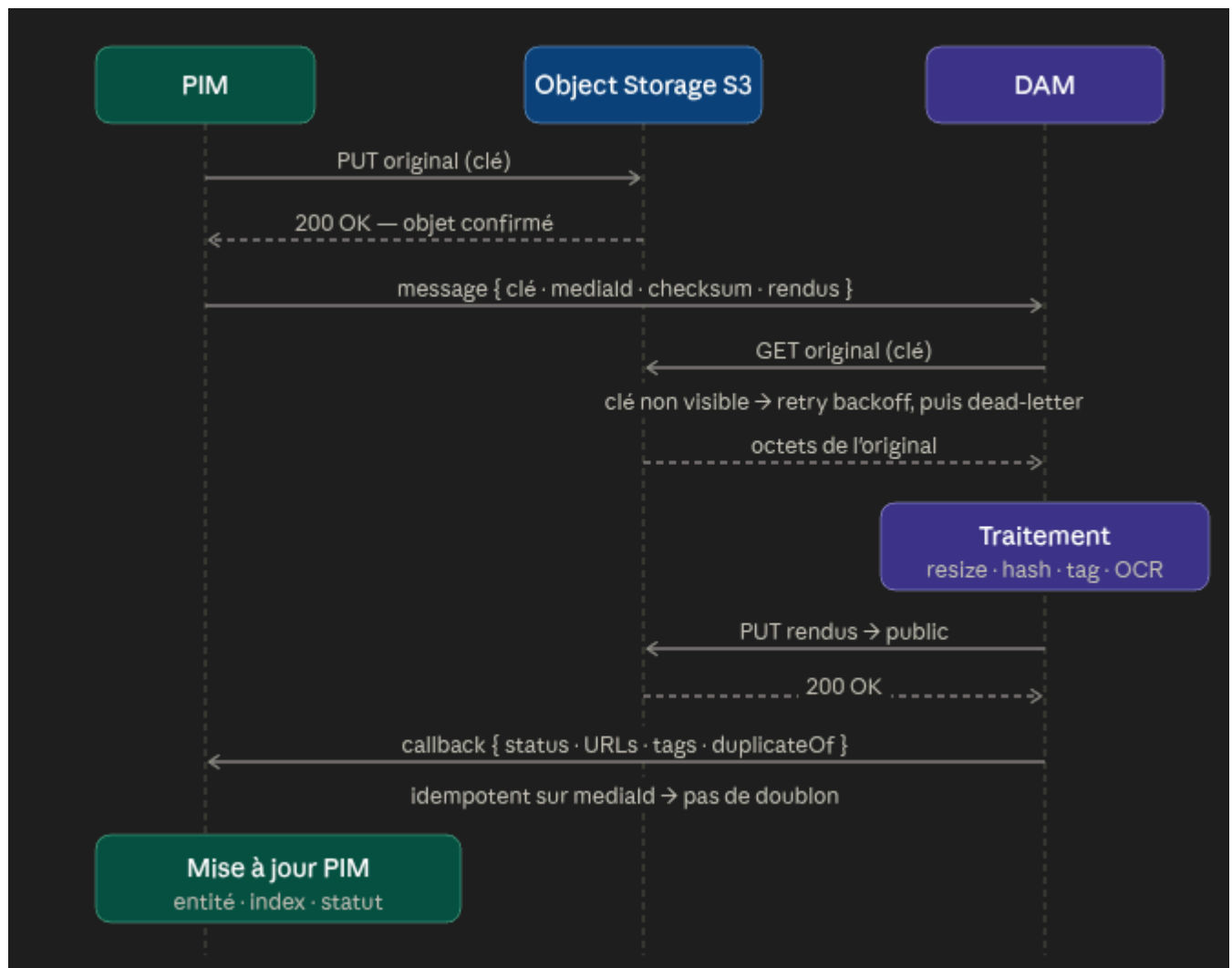
Upload → validation PIM → message Messenger → traitement DAM → rendus → CDN. Transport Doctrine en V1, RabbitMQ en V2 si le débit le nécessite.

5. Schémas d'architecture — traitements asynchrones



Files logiques Messenger dédiées par domaine. V1 : transport Doctrine/MariaDB. V2 : RabbitMQ et OpenSearch selon les déclencheurs mesurés.

5. Schémas d'architecture — séquence DAM



Dépôt S3, message, traitement et callback idempotent. Le contrat de message reste identique lors du passage du transport Doctrine à RabbitMQ.

6. Étapes de réalisation

Étape 1 — Socle V1

- Création d'un projet Upsun séparé de la Marketplace, avec serveur applicatif, base et workers distincts : environnements dev, staging et prod. Cette séparation limite les pannes communes liées à une application, un déploiement ou une saturation de ressources.
- Squelette Symfony avec structure modulaire, CI/CD, PHPStan et tests.
- Mise en place de MariaDB et de Symfony Messenger avec transport Doctrine, files logiques par domaine, workers et crons.
- Abstractions et configuration préparées pour RabbitMQ, Redis et OpenSearch, sans déployer ces services en V1.
- Sécurité Symfony : rôles BP et prestataires, voters limitant l'accès aux fiches affiliées.
- Gestion des utilisateurs : création Super Admin, anti-doublon email, activation et affiliation multi-fiches.

Étape 2 — PIM cœur

- Entité de base Fiche et cinq typologies : Lieu, Activité, Restaurant, Service événementiel et Traiteur/Plateau repas.
- Registre d'attributs : libellé, poids de complétude, caractère traduisible, règles conditionnelles et application maître.
- Back-office : création, modification, duplication, suppression, validation et champs conditionnels.
- Workflow : en cours → validé → publié → archivé.
- Complétude asynchrone et audit trail champ par champ en insertion seule.
- API REST via API Platform avec tokens et rate limiting.
- Alerte doublon d'adresse pour Lieux et Restaurants.
- Recherche MariaDB : index composites et fulltext sur nom, adresse et ID.

Étape 3 — DAM

- Buckets OVH pour les originaux et rendus, intégration Flysystem et CDN.
- Validation : formats PNG/JPG/JPEG/WEBP/PDF, poids selon typologie, dimensions minimales des images et rejets détaillés.
- Pipeline asynchrone : original → message Messenger → resize/WebP/vignettes → rendus publics → callback PIM.
- Hash exact et pHash, signalement non bloquant des doublons.
- Catégories, ordre par drag & drop, rattachement aux salles, édition, droits d'utilisation et URLs vidéo.

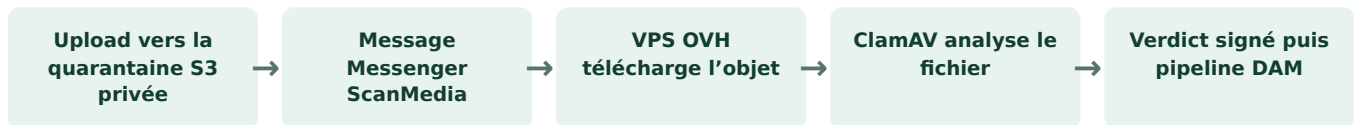
6. Étape à discuter — Antivirus des fichiers sur VPS OVH

Statut de cette proposition : je ne sais pas encore si une étape antivirus est réellement nécessaire pour la V1. Cette idée vient de **Claude Fable 5** et ne constitue pas, à ce stade, une exigence métier ou technique validée. Son intérêt, son coût et son niveau de priorité doivent être discutés avant toute mise en œuvre.

Si cette protection est retenue, le principe envisagé serait d'isoler l'analyse antivirus sur un **VPS OVH dédié**, afin de ne pas charger l'application Upsun et de conserver les fichiers non vérifiés hors du bucket public.

Principe envisagé

- Le fichier est d'abord déposé dans une zone S3 privée de quarantaine.
- Un antivirus ClamAV installé sur un VPS OVH analyse le fichier.
- Un fichier sain poursuit le pipeline DAM ; un fichier suspect reste isolé et déclenche une alerte.
- Si le scanner est indisponible, le fichier n'est pas publié tant que son statut n'est pas connu.



Décision à prendre avant la mise en production

- Confirmer si le risque justifie ce service supplémentaire.
- Préciser les formats concernés et la durée de conservation des fichiers suspects.
- Évaluer le coût du VPS, sa maintenance et l'impact sur le délai de publication d'un média.

6. Étapes de réalisation — suite

Étape 4 — Dashboards et supervision

- KPI : fiches publiées/en attente/totales, complétude, publications, activité, médias, stockage et champs peu renseignés.
- CRON de relance des fiches incomplètes et emails aux prestataires.
- Dashboard anomalies : doublons photos, incohérences et messages en échec.
- Suivi spécifique du transport Doctrine : profondeur des files, ancienneté du message le plus ancien et temps de traitement.

Étape 5 — ETL et synchronisations

- Import/export Excel/CSV/JSON découpé en jobs Messenger, avec rapports et historique.
- Connecteur Salesforce bidirectionnel : webhooks et réconciliation périodique.
- Autorité définie champ par champ ; le MDM reste le hub unique.
- Publication Marketplace BP, WordPress et API publiques.
- Retry avec backoff, files d'échec, logs et limitation de débit.

Étape 6 — IA

- Interface EnrichmentProvider indépendante des fournisseurs.
- Génération/reformulation avec statut provisoire et validation humaine.
- Traduction, tagging photo, OCR et extraction de données techniques.
- Google Business Profile, scraping et traçabilité de la provenance.

Étape 7 — Durcissement et mise en production

- Tests de charge progressifs jusqu'aux cibles de 200 000 fiches et 1 000 000 de médias.
- Mesure des recherches, imports, files Messenger, workers et requêtes API.
- Décision documentée d'activer ou non RabbitMQ, Redis et OpenSearch selon les seuils constatés.
- Logs centralisés, alertes, backups, restauration testée, sécurité et documentation.

7. Points de vigilance

- **Isolation d'hébergement** : le projet PIM/DAM dispose de ressources Upsun distinctes de la Marketplace. Cette séparation limite les pannes communes applicatives, sans prétendre couvrir une indisponibilité globale du fournisseur ou de la région.
- **Synchronisation bidirectionnelle** : application maître par champ et MDM en hub unique.
- **Fiabilité des messages** : handlers idempotents, retries, file d'échec et mécanisme outbox pour éviter l'écart entre transaction MariaDB et publication du message.
- **Transport Doctrine** : purger les messages terminés, indexer la table, surveiller la profondeur des files et limiter la concurrence. RabbitMQ devient prioritaire si la base métier est affectée.
- **Audit trail** : table en insertion seule et partitionnable.
- **Évolution des champs** : ajout par développement assumé afin de préserver Doctrine, Validator et API Platform.
- **IA** : aucun contenu généré n'est publié sans validation humaine.

8. V2 — services activables selon le besoin

RabbitMQ

À ajouter lorsque le transport Doctrine crée de la contention, lorsque les backlogs deviennent importants ou lorsqu'il faut faire monter les workers en charge indépendamment. Les messages et handlers construits en V1 sont conservés ; seuls le transport, le routage et le déploiement des consumers évoluent.

Redis

À ajouter pour partager un cache très sollicité entre plusieurs instances, centraliser les verrous, améliorer certains usages de session ou appliquer un rate limiting distribué. Les composants Symfony permettent de changer d'adaptateur sans réécrire les services métier.

OpenSearch

À ajouter pour les facettes complexes, la recherche multilingue avancée, un classement de pertinence élaboré ou lorsque les mesures démontrent que MariaDB ne respecte plus les objectifs de réponse. L'index sera alimenté de façon asynchrone par Messenger.

Autres options différées

- Data Governance Workspace complet après stabilisation des synchronisations.
- Création automatique de fiches par IA après validation du socle d'enrichissement.

Trajectoire retenue : V1 simple et exploitable avec Symfony, MariaDB, Messenger Doctrine et OVH S3/CDN ; V2 distribuée uniquement lorsque la charge réelle ou les besoins fonctionnels rendent RabbitMQ, Redis ou OpenSearch utiles.

9. Décisions restant à prendre

- Valider le périmètre fonctionnel exact de la V1.
- Confirmer l'hébergement PIM/DAM dans un projet Upsun distinct de la Marketplace.
- Décider si une protection antivirus dédiée est nécessaire dès la V1.
- Réévaluer RabbitMQ, Redis et OpenSearch après les premiers retours d'usage et tests de charge.

Conclusion : commencer avec une architecture Symfony simple sur un projet Upsun distinct, MariaDB et OVH S3/CDN, puis ajouter les services spécialisés uniquement lorsque leur besoin est constaté.